PRUEBA DE DESEMPEÑO

Pipeline ETL con Apache Airflow y Data Warehouse

Caso: DataMart S.A.S.

1. Contexto del negocio

DataMart S.A.S. es una empresa colombiana de comercio electrónico fundada en 2018. Opera en tres países: Colombia, México y Perú, y vende a través de dos canales principales: su tienda en línea y distribuidores minoristas aliados. Su catálogo tiene más de 3.000 referencias activas distribuidas en cinco categorías: Electrónica, Hogar, Ropa, Deportes y Papelería.

Durante 2023, la empresa experimentó un crecimiento del 40% en transacciones respecto al año anterior. Sin embargo, este crecimiento trajo un problema que hoy paraliza al equipo de inteligencia de negocio: los datos de ventas están en archivos planos que se generan cada día en el servidor de producción, el catálogo de productos se mantiene en un sistema legado sin una estructura analítica clara, y los datos de devolución y cancelación están mezclados con las ventas normales sin ninguna separación.

El equipo financiero no puede cerrar sus reportes mensuales sin hacer trabajo manual de consolidación que tarda entre tres y cinco días. El equipo de producto no sabe cuáles referencias están generando pérdidas. La dirección no puede comparar el desempeño entre países porque las monedas y los formatos de fecha son distintos en cada fuente.

La decisión estratégica ya fue tomada: DataMart va a construir su primera plataforma de datos. Tú eres el ingeniero de datos junior responsable de construir el primer pipeline de esa plataforma, el que va a demostrar que la arquitectura funciona y que los datos pueden fluir desde las fuentes operacionales hasta un repositorio donde el negocio pueda consultarlos.

2. Objetivo de la prueba

El objetivo es que construyas una solución funcional de extremo a extremo: desde la ingesta de los datos crudos hasta su disponibilidad en un repositorio analítico en PostgreSQL, todo orquestado con Apache Airflow corriendo dentro de Docker.

Se espera que puedas:

- Instalar y configurar Apache Airflow completamente dentro de Docker usando docker-compose, de forma que al ejecutar un solo comando el entorno quede listo para usarse sin ningún paso adicional.
- Conectar Airflow a las fuentes de datos y al repositorio analítico usando Airflow Connections, y controlar los parámetros operativos del pipeline usando Airflow Variables.
- Consumir tres fuentes de datos heterogéneas, explorarlas, identificar sus problemas de calidad y transformarlas aplicando las reglas de negocio que se describen en este documento.
- Construir un DAG de Airflow que orqueste el pipeline completo: extracción, transformación, validación de calidad y carga en el repositorio analítico.

- Diseñar la estructura del repositorio analítico en PostgreSQL según tu propio criterio, de forma que permita responder las preguntas de negocio planteadas.
- Documentar las decisiones que tomaste, incluyendo cómo manejaste los casos ambiguos que encontrarás en los datos.

3. Entorno Docker — Cómo debe quedar configurado

Airflow debe instalarse y correr únicamente a través de Docker. El repositorio debe incluir un archivo docker-compose.yml que levante todos los servicios con un solo comando: docker-compose up. Al ejecutarlo, deben quedar corriendo automáticamente y sin ninguna intervención manual:

- El webserver de Airflow, accesible en http://localhost:8080 con un usuario administrador ya creado.
- El scheduler de Airflow, que detecta y ejecuta los DAGs automáticamente.
- Una base de datos PostgreSQL dedicada a los metadatos internos de Airflow (no debe ser la misma que el repositorio analítico).
- Una base de datos PostgreSQL separada que funciona como repositorio analítico destino del pipeline, con las tablas ya creadas al iniciar.
- Los volúmenes o directorios compartidos necesarios para que Airflow acceda a los archivos CSV.

Además de los servicios, al levantar el entorno deben quedar configuradas automáticamente todas las Airflow Connections e inicializadas todas las Airflow Variables que el pipeline necesita para funcionar. Esto debe ocurrir como parte del proceso de inicio del contenedor, sin que sea necesario entrar a la UI de Airflow ni ejecutar comandos adicionales.

El repositorio debe incluir un archivo .env.example con todas las variables de entorno necesarias. El .env real debe estar excluido mediante .gitignore. Ningún usuario, contraseña ni cadena de conexión debe aparecer directamente en el código fuente.

4. Fuentes de datos

El pipeline debe procesar dos fuentes obligatorias: datasets de Kaggle que deberás incluir en el repositorio o descargar automáticamente. Existe además una tercera fuente opcional descrita al final de esta sección, cuya implementación suma valor pero no es requisito para que el pipeline funcione.

4.1 Transacciones de ventas (CSV diario)

Fuente: https://www.kaggle.com/datasets/carrie1/ecommerce-data — Archivo: data.csv

Este dataset contiene el registro transaccional de una tienda en línea del Reino Unido. Cada fila representa una línea de una factura e incluye el código del producto, la descripción, la cantidad, el precio unitario, la fecha, el identificador del cliente y el país. Para el contexto de DataMart, este archivo representa el volcado diario de órdenes del sistema operacional.

Al explorar este dataset encontrarás situaciones que requieren decisiones de negocio: hay transacciones con cantidades negativas que corresponden a devoluciones, hay códigos de producto que empiezan con letras que no siguen el patrón del catálogo, hay descripciones en mayúsculas mezcladas con otras en minúsculas para el mismo producto, y hay registros sin customer ID. No existe una única forma correcta de tratar cada caso: deberás tomar una decisión para cada uno y justificarla en tu documentación.

4.2 Historial extendido de transacciones (CSV)

Fuente: https://www.kaggle.com/datasets/thedevastator/online-retail-transaction-dataset — Archivo: online_retail_II.csv

Este dataset contiene dos años adicionales de transacciones del mismo tipo de negocio. Para DataMart representa el historial histórico que necesita cargar antes de activar el pipeline diario. Encontrarás que los formatos de fecha, los códigos de producto y los nombres de columna no son idénticos a los de la primera fuente, aunque representan el mismo tipo de información. Cómo decides unificar ambas fuentes, qué consideras compatible y qué no, es una decisión tuya que deberás documentar.

4.3 API interna de productos

DataMart mantiene su catálogo de productos en un sistema legado que expone una API REST. Si quieres ir más allá del requerimiento base, puedes construir esa API como un servicio adicional dentro del docker-compose, usando FastAPI o cualquier framework liviano de tu elección. La API debería exponer al menos dos endpoints: uno que devuelva el listado de productos con paginación, y otro que permita consultar un producto por su código. Si la construyes, cada producto debe tener al menos: código, nombre normalizado, categoría, país de origen del proveedor y si está activo o descontinuado.

El pipeline debe consumir la API y usar la información de categoría para enriquecer las transacciones. Si no la implementas, el pipeline debe igualmente tener una estrategia para asignar categoría a los productos — puede ser desde un archivo estático, desde los propios datos de las transacciones o desde cualquier otro enfoque que justifiques. El flujo del pipeline debe funcionar correctamente en ambos casos.

5. Reglas de negocio y casos de ambigüedad

A continuación se describen las reglas que DataMart ha definido para el pipeline, así como los casos ambiguos que encontrarás en los datos y que requerirán que tomes una decisión propia. Estas decisiones no tienen una respuesta única: lo que se espera es que tu elección sea coherente, aplicable de forma consistente a todos los registros y esté documentada.

Reglas definidas por el negocio

- Toda transacción con cantidad menor o igual a cero debe tratarse como devolución o ajuste, no como venta. El pipeline debe separarlas y registrarlas de forma que sea posible calcular el neto (ventas menos devoluciones) por producto y por periodo.
- El precio unitario no puede ser cero ni negativo en una venta válida. Los registros que incumplan esta condición deben registrarse en el log de rechazos con el motivo.
- Todos los campos de fecha deben estandarizarse a UTC antes de cargarlos al repositorio analítico.
- Los códigos de producto deben normalizarse a mayúsculas y sin espacios antes de cruzarlos con el catálogo de la API.
- El pipeline debe calcular y almacenar el revenue bruto (cantidad × precio unitario) y el revenue neto (revenue bruto después de aplicar cualquier ajuste por devolución asociada al mismo código de producto en el mismo periodo diario).

Casos ambiguos que deberás resolver

- Hay transacciones sin customer ID. El negocio no tiene una regla oficial sobre cómo tratarlas. Deberás decidir si las incluyes en el análisis, con qué valor de cliente, o si las excluyes, y documentar el impacto de esa decisión.
- Las descripciones de algunos productos tienen variaciones de escritura para el mismo código (CANDLE HOLDER WHITE, Candle Holder White, candle holder white). Deberás decidir cuál es el nombre canónico y cómo lo eliges.
- Los dos datasets de Kaggle se solapan en fechas en algunos registros. Deberás decidir cómo manejar los duplicados entre fuentes: ¿una fuente tiene prioridad sobre la otra, o usas alguna clave compuesta para detectar el duplicado real?

6. Alcance técnico — Qué se espera construir

A continuación se describe lo que se espera en cada parte del pipeline.

Exploración y limpieza

Antes de cargar cualquier dato al repositorio analítico, deberás explorar cada fuente y documentar lo que encontraste: estructura de columnas, tipos de dato, valores nulos, rangos, duplicados y distribuciones relevantes. Para cada problema de calidad, debes decidir si el registro se rechaza, se corrige o se trata de otra forma, y dejar esa lógica en el código de forma explicitamente legible. Los registros rechazados deben guardarse en una tabla de log dentro del repositorio analítico con el motivo y la fuente de origen.

Modelado del repositorio analítico

Una vez limpios los datos, deberás decidir cómo organizar la información en PostgreSQL: cuántas tablas, cómo se llaman, qué columnas tienen y cómo se relacionan. No se indica ninguna estructura esperada. El modelo debe ser coherente con las preguntas de negocio de la sección 7 y con las reglas de negocio de la sección 5, particularmente la separación entre ventas y devoluciones. Deberás justificar tu diseño en la documentación.

DAG de Airflow

El pipeline completo debe estar orquestado por un DAG con schedule diario. Ese DAG debe tener tareas separadas y con nombres descriptivos para cada etapa (extracción de cada fuente, transformación, carga), con dependencias explícitas entre ellas. Debe configurar reintentos automáticos ante fallos y ser idempotente: ejecutarlo dos veces el mismo día con los mismos datos debe producir exactamente el mismo resultado en el repositorio analítico. Deberás usar al menos una Airflow Connection para la base de datos destino y al menos dos Airflow Variables para parámetros operativos del pipeline.

7. Preguntas de negocio

El repositorio analítico que construyas debe ser capaz de responder estas preguntas. Para cada una deberás incluir en tu documentación al menos una consulta SQL que la responda usando los datos que cargaste.

- ¿Cuál fue la evolución mensual de las ventas netas (descontando devoluciones) durante el periodo cubierto por los datos?
- ¿Qué categorías de producto generaron más revenue bruto y cuáles tuvieron mayor proporción de devoluciones?
- ¿Cuáles son los 10 productos con mayor revenue neto y cuáles son los 10 con mayor tasa de devolución?

- ¿Qué países concentran la mayor parte de las transacciones y cómo varía el ticket promedio entre ellos?
- ¿Existe alguna diferencia en el comportamiento de compra entre clientes identificados y transacciones sin customer ID? (Esta pregunta solo aplica si decidiste incluir las transacciones sin cliente.)
- ¿Qué productos aparecen en las transacciones pero no tienen descripción consistente? ¿Cuántos códigos únicos de producto existen en total?
- Con base en los datos, ¿qué recomendación concreta y específica le harías al equipo de producto de DataMart? La recomendación debe estar respaldada por números del análisis, no ser genérica.

8. Entregables

Al finalizar las 8 horas deberás entregar los siguientes elementos a través del repositorio Git indicado por el evaluador.

Repositorio Git

El repositorio debe estar organizado de forma que refleje buenas prácticas: separación de responsabilidades entre archivos, nombres descriptivos, sin archivos temporales ni binarios innecesarios. Debe incluir como mínimo: el docker-compose.yml con todos los servicios, el .env.example, el código del DAG y sus dependencias, los scripts DDL de creación de tablas y los datos de prueba o seeds necesarios para ejecutar el pipeline desde cero.

README

El README debe permitir a alguien que nunca ha visto tu repositorio levantar el entorno y ejecutar el pipeline en menos de 10 minutos. Debe incluir los pasos exactos desde clonar el repositorio hasta verificar que los datos llegaron al repositorio analítico, incluyendo cómo validar que las Connections y Variables quedaron bien configuradas.

Documento de decisiones técnicas

Deberás redactar un documento (puede ser una sección del README o un archivo separado) en el que expliques: cómo diseñaste el modelo del repositorio analítico y por qué, cómo resolviste cada uno de los casos ambiguos de la sección 5, y cómo garantizas la idempotencia del DAG. Este documento es la evidencia más directa de tu proceso de pensamiento.

Diagrama del modelo de datos

Una imagen o enlace a dbdiagram.io que muestre las tablas que creaste en el repositorio analítico, sus columnas principales y las relaciones entre ellas.

Consultas SQL de validación

Al menos una consulta SQL por cada pregunta de la sección 7, ejecutable directamente contra el repositorio analítico después de correr el pipeline.

Sobre el tiempo disponible

Las 8 horas son suficientes para construir una solución funcional si priorizas bien. Empieza por levantar el entorno Docker y verificar que Airflow corra antes de escribir cualquier lógica de transformación. Si no alcanzas a implementar algo, docúmenta en el documento de decisiones qué quedaba pendiente y cómo lo habrías resuelto: esa documentación tiene tanto valor como el código.

¡Mucho éxito!